

第 10 卷 AI 专题与特判模型

考试速查：主查 AI 相关模拟题：搜索规划、KNN、相似度、TF-IDF、聚类、回归、HMM/Viterbi、SVM、DNN、反向传播、自动求导和 SPJ。

Contents

第 10 卷 AI 专题与特判模型	2
考试速查定位	2
AI 专项卷使用原则	2
AI-00 AI 专项可能题型路由	2
AI 题型路由表	3
AI-01 启发式搜索、路径规划与博弈 AI	5
Minimax / Alpha-Beta 怎么接	7
AI-02 轻量机器学习分类与评估	8
朴素贝叶斯路由	9
感知机路由	10
混淆矩阵	10
AI-03 相似度、推荐与文本向量	10
Top-K 推荐套路	13
文本向量套路	13
AI-04 AI 基础知识与公式速查	14
概念翻译表	15
常见公式	15
AI-05 文本处理、词频与 TF-IDF	16
AI-06 聚类、归一化与线性回归	18
线性回归路由	20
归一化路由	20
AI-07 Markov 链、HMM 与 Viterbi	21
AI-08 神经网络前向传播、激活函数与 Softmax	23
AI-09 强化学习、MDP 与值迭代	26
Q-learning 路由	27
AI-10 Special Judge、评分指标与模型选择策略	28
SPJ 考场 baseline 路由	29
验证集与调参策略	30
AI-11 线性 SVM、间隔分类与 Hinge Loss	30
AI-12 多层 DNN 前向传播模板	32
AI-13 线性回归与梯度下降训练	35
SPJ 调参建议	36
AI-14 反向传播、链式法则与小网络训练	37
多分类 softmax + 交叉熵规则	38
AI-15 计算图与反向模式自动求梯度	40
也可以封装成 AD 类	41

第 10 卷 AI 专题与特判模型

考试速查定位

第 10 卷 AI 专题与特判模型

- 这是第几卷： 10。
- 主要查什么：主查 AI 相关模拟题：搜索规划、KNN、相似度、TF-IDF、聚类、回归、HMM/Viterbi、SVM、DNN、反向传播、自动求导和 SPJ。
- 什么时候翻：题面带 AI/ML/聚类/回归/神经网络/自动求导/模型评测时翻。
- 题面关键词：搜索规划；KNN；TF-IDF；聚类；回归；Viterbi；DNN；SVM；反向传播；自动求导；SPJ。

自动由 AI 模块重建。定位是人工智能专项招生中可能包装成算法题的搜索、分类、相似度和模拟主题。

AI 专项卷使用原则

题面 AI 信号	算法本质
机器人/智能体/规划	BFS、Dijkstra、A*
对弈/智能决策	Minimax、Alpha-Beta、博弈 DP
训练集/测试集/标签	kNN、朴素贝叶斯、感知机、SVM、统计
Special Judge/得分函数	baseline、指标、验证集调参
相似文档/推荐	Cosine、Jaccard、Top-K
文本关键词/检索	TF-IDF、词频、倒排索引
聚类/回归	k-means、线性回归、梯度下降
Markov/HMM	Viterbi、概率 DP
神经网络推理/训练	前向传播、softmax、反向传播
计算图/自动求导	反向模式自动微分、链式法则
强化学习	MDP、值迭代、Q-learning 公式
脚本/规则/配置	SIM-03/04/05 解析和解释

这卷的核心判断：AI 背景不是新语法，也不是深度学习框架；它通常是搜索、统计、排序、字符串、模拟和数学公式的包装。

AI-00 AI 专项可能题型路由

模块编号：AI-00

模块名称：人工智能专项招生可能包装成机考题的算法主题

标签：AI、人工智能、搜索、机器学习、分类、相似度、模拟、C++17

一句话用途：如果题面带“智能体、训练数据、分类、相似度、推荐、路径规划、博弈、规则引擎”等 AI 味道，先用这张表判断它本质上是哪类算法题。

题面触发词：

- 智能体、机器人、路径规划、启发式、代价函数。
- 游戏 AI、对手、最优策略、轮流行动。
- 训练集、测试集、标签、分类、预测。
- 特征向量、距离、相似度、推荐。
- 准确率、召回率、混淆矩阵。
- 文本分词、词频、关键词、相似文档。
- 简单神经元、权重、训练轮数、梯度下降。
- Special Judge、得分、误差、模型选择。
- SVM、margin、hinge loss。
- DNN、多层网络、前向传播、反向传播。
- 计算图、自动求导、gradient、chain rule。

什么时候用：

- 题目是 AI 背景，但实际可化为搜索、图、数学、模拟、排序或 DP。
- 需要快速判断“是不是要写机器学习”，避免被题面吓住。
- 需要无第三方库手写 kNN、朴素贝叶斯、感知机、相似度等轻量模型。

不要什么时候用：

- 不要准备深度学习框架；但要准备小规模反向传播/计算图求导模板，用来应对规则模拟题。
- 不要把 AI 题理解成要懂很多理论；大概率仍是基础算法/数据结构包装。
- 如果数据规模很大，轻量 ML 模拟也可能变成排序、矩阵、前缀统计或图题。

复杂度：

- AI 包装题仍按算法复杂度判断。
- kNN: $O(\text{测试数} * \text{训练数} * \text{维度})$ 。
- 朴素贝叶斯: 训练 $O(\text{样本数} * \text{特征数})$ ，预测 $O(\text{类别数} * \text{特征数})$ 。
- A*: 最坏仍可能接近 Dijkstra/BFS 访问状态数。
- Minimax: 约 $O(\text{分支数}^{\text{深度}})$ ，alpha-beta 只剪枝不改变最坏指数级。

依赖的标准容器：

- vector: 特征、样本、距离、矩阵。
- string: 标签、文本、token。
- map/unordered_map: 标签计数、词频、词典。
- priority_queue: A*、Top-K 推荐。
- queue/deque: BFS 状态搜索。

输入如何整理：

```
struct Sample {
    vector<double> x;
    int label;
};

int n, d;
cin >> n >> d;
vector<Sample> train(n + 1);
for (int i = 1; i <= n; i++) {
    train[i].x.assign(d + 1, 0);
    for (int j = 1; j <= d; j++) cin >> train[i].x[j];
    cin >> train[i].label;
}
```

接口：

AI 题面 -> 算法本质：

路径规划 -> BFS / Dijkstra / A*

对弈决策 -> minimax / alpha-beta / 博弈 DP

分类预测 -> kNN / Naive Bayes / Perceptron / SVM

相似推荐 -> cosine / Jaccard / Top-K

评估指标 -> confusion matrix / accuracy / precision / recall

文本处理 -> 分词、词频、哈希、字符串

规则执行 -> SIM-03/04/05 解析器和解释器

SPJ 得分 -> baseline / metric / validation

计算图求导 -> forward / reverse backward

AI 题型路由表

AI 题面	本质模块	优先翻
机器人从起点到终点，格子有障碍/代价	图搜索	GRAPH-02/03, AI-01
启发式函数、估价函数、曼哈顿距离	A*	AI-01
双方轮流，最优行动	博弈搜索 / DP	AI-01, DP-20
给训练集和测试集，按距离分类	kNN	AI-02
特征独立、条件概率、文本分类	朴素贝叶斯	AI-02
权重、学习率、训练轮数	感知机/线性模型	AI-02
SVM、margin、hinge loss	线性 SVM	AI-11
文档相似、用户相似、向量夹角	cosine/Jaccard	AI-03
Top-K 推荐	排序/堆	AI-03, CPP-004
分词、词频、关键词	字符串 + map	AI-03, STR-01
混淆矩阵、准确率、召回率	模拟统计	AI-02/03
Special Judge、误差、得分函数	指标 + baseline + 调参	AI-10
解析规则/脚本/配置	解析器/解释器	SIM-03/04/05
TF-IDF、关键词检索	文本向量	AI-05

AI 题面	本质模块	优先翻
聚类中心、k-means	无监督聚类	AI-06
连续值预测、MSE、gradient descent	线性回归	AI-13
Markov、HMM、观测序列	概率 DP	AI-07
神经网络前向传播	矩阵向量公式	AI-08/12
反向传播、链式法则、参数更新	小网络训练模拟	AI-14
计算图、自动求导、偏导	反向模式自动微分	AI-15
状态、动作、奖励、策略	MDP / RL	AI-09

模板代码:

```
#include <bits/stdc++.h>
using namespace std;

string route_ai_topic(const string &word) {
    if (word == "path" || word == "robot" || word == "astar") return "AI-01 / GRAPH";
    if (word == "game" || word == "minimax") return "AI-01 / DP";
    if (word == "knn" || word == "classify") return "AI-02";
    if (word == "similarity" || word == "recommend") return "AI-03";
    if (word == "spj" || word == "score" || word == "metric") return "AI-10";
    if (word == "svm" || word == "margin") return "AI-11";
    if (word == "dnn" || word == "forward") return "AI-12";
    if (word == "regression" || word == "mse") return "AI-13";
    if (word == "backprop" || word == "backward") return "AI-14";
    if (word == "autograd" || word == "gradient") return "AI-15";
    if (word == "json" || word == "script") return "SIM-04/05";
    return "ROUTE";
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string word;
    while (cin >> word) {
        cout << route_ai_topic(word) << '\n';
    }
    return 0;
}
```

调用示例:

```
cout << route_ai_topic("knn") << '\n';
cout << route_ai_topic("robot") << '\n';
```

常见坑:

- AI 背景不等于要写复杂 AI; 先翻译成算法关键词。
- 数据规模决定语言和算法, 不由题目背景决定。
- 没有第三方库时, 不要幻想 numpy、模型库、矩阵库。
- 小机器学习题常常就是“按公式模拟 + 排序/统计”。
- A* 的启发式必须不能高估真实代价, 才能保证最短路正确。

暴力/部分替代:

- 路径规划不会 A*: 先 BFS/Dijkstra。
- 博弈不会剪枝: 先 DFS 暴力小深度, 再加记忆化。
- 分类不会模型: 先多数类、最近 1 个样本、简单距离。
- 推荐不会复杂算法: 先按共同物品数、Jaccard 或 cosine 排序。
- 文本题不会高级 NLP: 先 split、词频、停用词特判。

最小测试样例:

```
输入
knn
robot
```

script

输出

AI-02

AI-01 / GRAPH

SIM-04/05

AI-01 启发式搜索、路径规划与博弈 AI

模块编号: AI-01

模块名称: A* 路径规划、启发式搜索与 Minimax 思想

标签: AI、A*、启发式搜索、路径规划、游戏 AI、Minimax、Alpha-Beta、C++17

一句话用途: AI 背景中的“机器人找路”和“游戏最优策略”通常就是图搜索和博弈搜索,本模块给出可直接抄的 A* 网格模板和 minimax 路由。

题面触发词:

- 机器人、智能体、地图、起点、终点、障碍、路径规划。
- 启发式函数、估价函数、曼哈顿距离。
- 游戏 AI、双方轮流、当前玩家、最优行动。
- 搜索树、剪枝、估值函数。

什么时候用:

- 网格每步代价相同或非负,且有明确目标点。
- 需要比普通 BFS/Dijkstra 更像 AI 路径规划的写法。
- 博弈题状态规模小,可以搜索到终局或限定深度。

不要什么时候用:

- 无权最短路普通 BFS 已足够时,不要为了 AI 背景硬写 A*。
- 启发式可能高估真实距离时, A* 不保证最短路。
- 博弈状态大且有明显 DP 结构时,优先状态 DP/记忆化。
- 有负边权最短路不要用 A* 或 Dijkstra。

复杂度:

- A* 最坏仍可能访问大量状态,通常不超过 Dijkstra 数量级。
- 网格 A* 每个格子入堆若干次,常写成 $O(n*m*\log(n*m))$ 。
- Minimax 不剪枝约 $O(b^d)$, b 是分支数, d 是搜索深度。

依赖的标准容器:

- `vector<string>`: 1-index 网格。
- `priority_queue`: A* open set。
- `vector<vector<int>>`: 距离数组。
- `map/unordered_map`: 复杂状态记忆化。

输入如何整理:

```
int n, m;
cin >> n >> m;
vector<string> grid(n + 1);
for (int i = 1; i <= n; i++) {
    string row;
    cin >> row;
    grid[i] = " " + row; // 1-index 列
}
```

接口:

`astar_grid(grid, n, m, sx, sy, tx, ty) -> 最短步数, 不可达返回 -1。`

启发式 $h(x,y)=abs(x-tx)+abs(y-ty)$ 。

模板代码:

```

#include <bits/stdc++.h>
using namespace std;

struct State {
    int f;
    int g;
    int x;
    int y;
    bool operator<(const State &other) const {
        if (f != other.f) return f > other.f;
        return g > other.g;
    }
};

int astar_grid(const vector<string> &grid, int n, int m, int sx, int sy, int tx, int ty) {
    const int INF = 1e9;
    vector<vector<int>> dist(n + 1, vector<int>(m + 1, INF));
    priority_queue<State> pq;

    auto inside = [&](int x, int y) {
        return 1 <= x && x <= n && 1 <= y && y <= m && grid[x][y] != '#';
    };
    auto h = [&](int x, int y) {
        return abs(x - tx) + abs(y - ty);
    };

    dist[sx][sy] = 0;
    pq.push({h(sx, sy), 0, sx, sy});

    int dx[5] = {0, -1, 1, 0, 0};
    int dy[5] = {0, 0, 0, -1, 1};

    while (!pq.empty()) {
        State cur = pq.top();
        pq.pop();
        if (cur.g != dist[cur.x][cur.y]) continue;
        if (cur.x == tx && cur.y == ty) return cur.g;

        for (int dir = 1; dir <= 4; dir++) {
            int nx = cur.x + dx[dir];
            int ny = cur.y + dy[dir];
            if (!inside(nx, ny)) continue;
            int ng = cur.g + 1;
            if (ng < dist[nx][ny]) {
                dist[nx][ny] = ng;
                pq.push({ng + h(nx, ny), ng, nx, ny});
            }
        }
    }

    return -1;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    cin >> n >> m;
    vector<string> grid(n + 1);
    int sx = -1, sy = -1, tx = -1, ty = -1;

    for (int i = 1; i <= n; i++) {

```

```

    string row;
    cin >> row;
    grid[i] = " " + row;
    for (int j = 1; j <= m; j++) {
        if (grid[i][j] == 'S') {
            sx = i;
            sy = j;
        } else if (grid[i][j] == 'T') {
            tx = i;
            ty = j;
        }
    }
}

if (sx == -1 || tx == -1) {
    cout << -1 << '\n';
    return 0;
}

cout << astar_grid(grid, n, m, sx, sy, tx, ty) << '\n';
return 0;
}

```

Minimax / Alpha-Beta 怎么接

概念骨架:

score(state, player):

- 如果终局, 返回当前局面对“我方”的分数
- 如果轮到我方, 取所有后继的最大分
- 如果轮到对方, 取所有后继的最小分

能加速的地方:

- 终局判断。
- 估价函数: 深度到头时用局面评分。
- Alpha-Beta: 当前分支已经不可能更优就剪掉。
- 记忆化: 同一个状态和当前玩家不重复算。

调用示例:

```

// 网格路径规划:
// int ans = astar_grid(grid, n, m, sx, sy, tx, ty);

```

常见坑:

- A^* 的 $f = g + h$, g 是已走代价, h 是估计剩余代价。
- 曼哈顿距离适合四方向无障碍估计; 有障碍也不高估, 仍可用。
- 如果每步代价不同, g 要加真实边权, h 也不能高估。
- 如果 $h=0$, A^* 退化成 Dijkstra。
- 博弈题要明确评分是从谁的视角。
- 记忆化 key 必须包含当前玩家/轮次, 否则会错。

暴力/部分替代:

- A^* 不会写: 无权先 BFS, 非负权先 Dijkstra。
- 启发式不会设计: 令 $h=0$, 至少正确。
- Minimax 不会剪枝: 先 DFS 限深。
- 状态重复多: 加 $\text{map}<\text{state}, \text{int}>$ 记忆化。
- 对局太大: 只搜索小深度, 用估价函数拿部分分。

最小测试样例:

```

输入
3 4
S..#
.#..
...T

```

AI-02 轻量机器学习分类与评估

模块编号: AI-02

模块名称: kNN、朴素贝叶斯、感知机与混淆矩阵路由

标签: AI、机器学习、分类、kNN、朴素贝叶斯、感知机、混淆矩阵、C++17

一句话用途: 当题目给训练集、测试集、标签和特征时, 把它当作“按公式模拟 + 排序/统计”的算法题, 优先使用 kNN、计数概率或线性打分。

题面触发词:

- 训练集、测试集、样本、标签、特征。
- 最近邻、距离、分类。
- 条件概率、先验概率、词频、文本分类。
- 权重、学习率、训练轮数、预测正负类。
- 准确率、混淆矩阵、precision、recall。

什么时候用:

- 数据规模中小, 直接按训练集扫描每个测试样本可接受。
- 题目明确给出 k 、距离公式或分类规则。
- 特征是离散计数, 可以做朴素贝叶斯。
- 二分类线性模型按题面给定公式训练。

不要什么时候用:

- 不要实现复杂神经网络、反向传播和矩阵库。
- 数据极大时, kNN 的 测试数 * 训练数 * 维度 会 TLE。
- 浮点误差敏感时, 比较要按题目要求设置精度。
- 类别很多且概率极小, 朴素贝叶斯要用 $\log$, 不能直接乘很多小数。

复杂度:

- kNN 预测一个样本: $O(n*d + n \log n)$, 可用 `nth_element` 降到均摊 $O(n*d + n)$ 。
- 混淆矩阵: $O(q)$ 。
- 朴素贝叶斯训练: $O(n*d)$, 预测: $O(\text{class_count}*d)$ 。
- 感知机: $O(\text{epoch}*n*d)$ 。

依赖的标准容器:

- `vector<double>`: 特征。
- `vector<int>`: 标签。
- `map<int,int>`: 投票计数。
- `vector<vector<int>>`: 混淆矩阵。

输入如何整理:

```
struct Sample {
    vector<double> x; // 1-index 特征
    int label;
};
```

接口:

`predict_knn(train, query, k, d) -> 预测标签。`

tie-break: 票数多优先; 票数相同标签小优先。

模板代码:

```
#include <bits/stdc++.h>
using namespace std;

struct Sample {
    vector<double> x;
    int label;
};
```



```

double squared_distance(const vector<double> &a, const vector<double> &b, int d) {
    double res = 0;
    for (int i = 1; i <= d; i++) {
        double diff = a[i] - b[i];
        res += diff * diff;
    }
    return res;
}

int predict_knn(const vector<Sample> &train, const vector<double> &query, int k, int d) {
    vector<pair<double, int>> dist;
    for (int i = 1; i < (int)train.size(); i++) {
        dist.push_back({squared_distance(train[i].x, query, d), train[i].label});
    }
    sort(dist.begin(), dist.end());

    map<int, int> vote;
    for (int i = 0; i < k && i < (int)dist.size(); i++) {
        vote[dist[i].second]++;
    }

    int best_label = -1;
    int best_count = -1;
    for (auto [label, count] : vote) {
        if (count > best_count || (count == best_count && label < best_label)) {
            best_count = count;
            best_label = label;
        }
    }
    return best_label;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, q, d, k;
    cin >> n >> q >> d >> k;

    vector<Sample> train(n + 1);
    for (int i = 1; i <= n; i++) {
        train[i].x.assign(d + 1, 0);
        for (int j = 1; j <= d; j++) cin >> train[i].x[j];
        cin >> train[i].label;
    }

    for (int i = 1; i <= q; i++) {
        vector<double> query(d + 1);
        for (int j = 1; j <= d; j++) cin >> query[j];
        cout << predict_knn(train, query, k, d) << '\n';
    }

    return 0;
}

```

朴素贝叶斯路由

适合离散特征或词频文本分类。核心公式：

$$\text{score}[c] = \log(P(c)) + \sum \log(P(\text{feature_j} \mid c))$$

为什么用 log：

- 很多小概率直接相乘会下溢。

- $\log$ 后乘法变加法，比较大小不变。

感知机路由

适合二分类、题面直接给训练轮数和学习率：

```
pred = sign(w · x + b)
如果 pred != y:
    w = w + lr * y * x
    b = b + lr * y
```

标签通常要转成 -1/+1。

混淆矩阵

```
matrix[真实标签][预测标签]++
accuracy = 正确数 / 总数
```

调用示例：

```
// int label = predict_knn(train, query, k, d);
```

常见坑：

- kNN 距离可以不开根号，平方距离排序结果相同。
- 浮点排序相等时，要按标签或样本编号做稳定 tie-break。
- 标签不一定从 1 连续到 c，可先离散化。
- k 大于训练样本数时，只取所有训练样本。
- 朴素贝叶斯要做平滑，例如 +1，避免概率为 0。
- 精度输出用 `fixed << setprecision(...)`。

暴力/部分分替代：

- kNN 太慢：先支持 $k=1$ 或测试数小的子任务。
- 朴素贝叶斯不会：先按每类样本数预测多数类。
- 感知机不会：按题面给的权重直接预测，不训练。
- 混淆矩阵不会指标：先输出 accuracy 或正确数。

最小测试样例：

输入

```
4 3 2 3
0 0 1
0 1 1
10 10 2
10 11 2
0 0.2
9 10
5 5
```

输出

```
1
2
1
```

AI-03 相似度、推荐与文本向量

模块编号：AI-03

模块名称：Cosine、Jaccard、混淆矩阵与 Top-K 推荐

标签：AI、相似度、推荐、文本、向量、Cosine、Jaccard、Top-K、C++17

一句话用途：当题目要求计算用户相似、文档相似、向量相似、推荐 Top-K 或评估预测结果时，用本模块把 AI 背景化成排序、集合和向量运算。

题面触发词：

- 相似度、余弦相似度、夹角、向量。
- Jaccard、交集、并集、共同兴趣。

- 推荐、Top-K、最相似用户、最相关文档。
- 词频、关键词、文本向量。
- 准确率、混淆矩阵、预测标签。

什么时候用：

- 特征已经给成向量或集合。
- 文本可以按空格切词统计词频。
- 推荐规则就是按相似度排序。
- 指标按公式计算，不需要真正训练模型。

不要什么时候用：

- 不要实现复杂深度语义模型、词向量训练、神经网络。
- 文本分词如果涉及中文自然语言复杂规则，按题面给的分词规则走。
- 向量维度和样本数都很大时，注意 $O(n*d)$ 是否可承受。

复杂度：

- cosine: $O(d)$ 。
- Jaccard: 排序后 $O(n+m)$ ，或 set 统计。
- Top-K: 全排序 $O(n \log n)$ ，堆可 $O(n \log k)$ 。
- 混淆矩阵: $O(q)$ 。

依赖的标准容器：

- `vector<double>`: 向量。
- `vector<int>`: 集合元素、标签。
- `map<string,int>`: 词频。
- `priority_queue`: Top-K。

输入如何整理：

```
int d;
cin >> d;
vector<double> a(d + 1), b(d + 1);
for (int i = 1; i <= d; i++) cin >> a[i];
for (int i = 1; i <= d; i++) cin >> b[i];
```

接口：

`cosine(a,b,d)` -> 余弦相似度。

`jaccard(A,B)` -> 集合 Jaccard，相同元素先去重。

模板代码：

```
#include <bits/stdc++.h>
using namespace std;

double cosine_similarity(const vector<double> &a, const vector<double> &b, int d) {
    double dot = 0;
    double na = 0;
    double nb = 0;
    for (int i = 1; i <= d; i++) {
        dot += a[i] * b[i];
        na += a[i] * a[i];
        nb += b[i] * b[i];
    }
    if (na == 0 || nb == 0) return 0;
    return dot / sqrt(na * nb);
}

double jaccard_similarity(vector<int> a, vector<int> b) {
    sort(a.begin() + 1, a.end());
    sort(b.begin() + 1, b.end());
    a.erase(unique(a.begin() + 1, a.end()), a.end());
    b.erase(unique(b.begin() + 1, b.end()), b.end());

    int i = 1;
    int j = 1;
```

```

int inter = 0;
int uni = 0;

while (i < (int)a.size() || j < (int)b.size()) {
    if (j >= (int)b.size() || (i < (int)a.size() && a[i] < b[j])) {
        uni++;
        i++;
    } else if (i >= (int)a.size() || b[j] < a[i]) {
        uni++;
        j++;
    } else {
        inter++;
        uni++;
        i++;
        j++;
    }
}

if (uni == 0) return 0;
return (double)inter / uni;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string mode;
    cin >> mode;

    cout << fixed << setprecision(6);
    if (mode == "cosine") {
        int d;
        cin >> d;
        vector<double> a(d + 1), b(d + 1);
        for (int i = 1; i <= d; i++) cin >> a[i];
        for (int i = 1; i <= d; i++) cin >> b[i];
        cout << cosine_similarity(a, b, d) << '\n';
    } else if (mode == "jaccard") {
        int n, m;
        cin >> n >> m;
        vector<int> a(n + 1), b(m + 1);
        for (int i = 1; i <= n; i++) cin >> a[i];
        for (int i = 1; i <= m; i++) cin >> b[i];
        cout << jaccard_similarity(a, b) << '\n';
    } else if (mode == "confusion") {
        int c, q;
        cin >> c >> q;
        vector<vector<int>> mat(c + 1, vector<int>(c + 1, 0));
        int correct = 0;
        for (int i = 1; i <= c; i++) {
            int real_label, pred_label;
            cin >> real_label >> pred_label;
            mat[real_label][pred_label]++;
            if (real_label == pred_label) correct++;
        }
        cout << (double)correct / q << '\n';
        for (int i = 1; i <= c; i++) {
            for (int j = 1; j <= c; j++) {
                if (j > 1) cout << ' ';
                cout << mat[i][j];
            }
            cout << '\n';
        }
    }
}

```

```

    }

    return 0;
}

```

Top-K 推荐套路

对每个候选 item 计算 score
按 score 降序、id 升序排序
输出前 k 个

如果候选很多：

priority_queue 保留前 k 个

文本向量套路

1. 按题面规则切词。
2. map<string,int> 统计词频。
3. 统一词典后变成向量。
4. 用 cosine/Jaccard 比较。

调用示例：

```

// double sim = cosine_similarity(a, b, d);
// double jac = jaccard_similarity(A, B);

```

常见坑：

- cosine 分母为 0 时要防御。
- Jaccard 要先去重；多重集合相似度是另一种题。
- 相似度排序要写清楚 tie-break，常见是分数高优先、id 小优先。
- 文本大小写、标点、停用词都按题面规则处理，不要自行脑补。
- double 输出按题目要求设置精度。

暴力/部分分替代：

- 推荐不会优化：先全排序。
- 文本不会建完整向量：先统计共同词数。
- cosine 不会：先用点积排序，有些数据可拿部分分。
- 混淆矩阵不会高级指标：先输出矩阵和 accuracy。

最小测试样例：

输入

```

cosine
3
1 0 1
1 1 0

```

输出

```

0.500000

```

补充自测：

输入

```

jaccard
4 5
1 2 2 3
2 3 4 4 5

```

输出

```

0.400000

```

补充自测 2：

输入

```

confusion
3 5
1 1

```

```
1 2
2 2
3 1
3 3

输出
0.600000
1 1 0
0 1 0
1 0 1
```

AI-04 AI 基础知识与公式速查

模块编号: AI-04

模块名称: AI 必备基础概念、常见公式与机考落地方式

标签: AI、机器学习、监督学习、无监督学习、强化学习、损失函数、评估指标、C++17

一句话用途: 遇到 AI 专项背景题时, 先用这一页把术语翻译成可手写算法和公式, 避免看到“模型、训练、推理、损失、策略”就卡住。

题面触发词:

- 数据集、训练集、测试集、特征、标签。
- 模型、参数、权重、偏置、学习率、迭代轮数。
- 损失函数、准确率、召回率、F1。
- 反向传播、计算图、自动求导、链式法则。
- 聚类、中心、距离、相似度。
- 状态、动作、奖励、策略、价值。

什么时候用:

- 题目用了 AI/ML 术语, 但没有要求第三方库。
- 需要把题面公式转成循环和数组。
- 需要确认某个指标、模型或训练过程的基本含义。

不要什么时候用:

- 不要把这页当机器学习教材; 机考通常只考公式模拟和基础算法。
- 不要准备深度学习框架、复杂矩阵库; 但要会按题面规则模拟小计算图、反向传播和自动求导。
- 如果题面给了具体公式, 以题面公式为准。

复杂度:

- 训练/推理复杂度按样本数、特征数、迭代次数估算。
- 典型公式: $O(\text{epoch} * n * d)$ 、 $O(\text{test} * \text{train} * d)$ 、 $O(\text{iter} * n * k * d)$ 。

依赖的标准容器:

- `vector<double>`: 特征、权重、中心。
- `vector<int>`: 标签、类别、动作。
- `map/unordered_map`: 词典、类别计数。
- `priority_queue`: Top-K。

输入如何整理:

```
int n, d;
cin >> n >> d;
vector<vector<double>>> x(n + 1, vector<double>(d + 1));
vector<int> y(n + 1);
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= d; j++) cin >> x[i][j];
    cin >> y[i];
}
```

接口:

监督学习: 有标签 `y`, 做分类/回归。

无监督学习: 无标签 `y`, 做聚类/降维/相似度。

强化学习：有状态、动作、奖励，做策略或价值更新。

推理：给定模型参数，算输出。

训练：按题面规则迭代更新参数。

概念翻译表

AI 术语	竞赛题里的含义	翻模块
feature	一个样本的数组	vector<double>
label	类别编号或目标值	int/double
inference	给参数算答案	循环公式
training	重复更新参数	模拟迭代
loss	误差函数	数学公式
gradient	更新方向	AI-14/15 或按题面公式
classification	输出类别	AI-02/08/11/12
regression	输出连续值	AI-06/13
clustering	分组	AI-06
policy	状态到动作	AI-09
value	状态未来收益	AI-09
special judge	按分数评测	AI-10
backpropagation	从输出层往前传梯度	AI-14
autograd	计算图反向求偏导	AI-15

常见公式

```
dot(w,x) = sum w[i] * x[i]
linear = dot(w,x) + b
relu(x) = max(0,x)
sigmoid(x) = 1 / (1 + exp(-x))
softmax[i] = exp(z[i]) / sum exp(z[j])
MSE = average((pred - y)^2)
hinge_loss = max(0, 1 - y * score)
accuracy = correct / total
precision = TP / (TP + FP)
recall = TP / (TP + FN)
F1 = 2 * precision * recall / (precision + recall)
chain_rule: dL/dx = dL/dy * dy/dx
```

模板代码：

```
#include <bits/stdc++.h>
using namespace std;

double dot_product(const vector<double> &a, const vector<double> &b, int d) {
    double res = 0;
    for (int i = 1; i <= d; i++) res += a[i] * b[i];
    return res;
}

double relu(double x) {
    return max(0.0, x);
}

double sigmoid(double x) {
    return 1.0 / (1.0 + exp(-x));
}

double f1_score(double tp, double fp, double fn) {
    double precision = (tp + fp == 0) ? 0 : tp / (tp + fp);
    double recall = (tp + fn == 0) ? 0 : tp / (tp + fn);
    if (precision + recall == 0) return 0;
    return 2 * precision * recall / (precision + recall);
}
```

调用示例:

```
double z = dot_product(w, x, d) + b;  
double y = sigmoid(z);
```

常见坑:

- 题目给的标签可能是 0/1, 也可能是 -1/+1, 训练公式不同。
- softmax 要减最大值防止 exp 溢出。
- 浮点输出按题目要求 fixed << setprecision(k)。
- 分类 tie-break 通常要按标签小、编号小或输入顺序。
- “训练轮数” 是外层循环, 不要少跑或多跑一轮。

暴力/部分替代:

- 不会训练: 先实现推理。
- 不会复杂模型: 先实现多数类、最近邻或线性打分。
- 不会优化: 先按题面公式逐样本模拟。

最小测试样例:

本模块是公式速查, 无完整输入输出主程序。

AI-05 文本处理、词频与 TF-IDF

模块编号: AI-05

模块名称: 文本分词、词频统计、TF-IDF 与文档相似度

标签: AI、NLP、文本处理、TF-IDF、词频、文档相似度、C++17

一句话用途: 当题目给文档、查询、关键词或相似文本时, 用分词 + 词频 + TF-IDF + cosine 把自然语言题变成字符串和向量题。

题面触发词:

- 文档、查询、关键词、词频、倒排索引。
- TF、IDF、TF-IDF。
- 文本相似度、最相关文档。
- 忽略大小写、去掉标点、按非字母数字切词。

什么时候用:

- 题面给了英文文本或已经分好的 token。
- 需要统计词频、文档频率、关键词分数。
- 要按查询文本找最相似文档。

不要什么时候用:

- 中文复杂分词、语义理解、词向量训练, 不适合手写。
- 题面如果给了自己的分词规则, 以题面为准。
- 文档极大时, 要注意 map 和字符串内存。

复杂度:

- 分词: $O(\text{总字符数})$ 。
- 统计词频: $O(\text{总 token 数} * \log \text{词典})$, 用 map。
- 查询所有文档: $O(\text{文档数} * \text{查询词数})$ 或按实现略高。

依赖的标准容器:

- string
- vector<string>
- map<string,int>
- set<string>
- vector<map<string,int>>

输入如何整理:

```
int n;  
cin >> n;  
string line;  
getline(cin, line);
```



```
for (int i = 1; i <= n; i++) getline(cin, doc[i]);
getline(cin, query);
```

接口:

tokenize(line) -> 小写 token 列表。

tfidf_cosine(doc_count, query_count, df, n) -> 相似度。

模板代码:

```
#include <bits/stdc++.h>
using namespace std;

vector<string> tokenize(const string &s) {
    vector<string> words;
    string cur;
    for (char ch : s) {
        unsigned char c = (unsigned char)ch;
        if (isalnum(c)) {
            cur.push_back((char)tolower(c));
        } else if (!cur.empty()) {
            words.push_back(cur);
            cur.clear();
        }
    }
    if (!cur.empty()) words.push_back(cur);
    return words;
}

map<string, int> count_words(const vector<string> &words) {
    map<string, int> cnt;
    for (const string &w : words) cnt[w]++;
    return cnt;
}

double tfidf_cosine(const map<string, int> &a, const map<string, int> &b, const map<string, int> &df, int n) {
    map<string, double> va, vb;
    for (auto [w, c] : a) {
        int dfi = df.count(w) ? df.at(w) : 0;
        double idf = log((double)(n + 1) / (dfi + 1)) + 1;
        va[w] = c * idf;
    }
    for (auto [w, c] : b) {
        int dfi = df.count(w) ? df.at(w) : 0;
        double idf = log((double)(n + 1) / (dfi + 1)) + 1;
        vb[w] = c * idf;
    }

    double dot = 0;
    double na = 0;
    double nb = 0;
    for (auto [w, x] : va) {
        na += x * x;
        if (vb.count(w)) dot += x * vb[w];
    }
    for (auto [w, y] : vb) nb += y * y;
    if (na == 0 || nb == 0) return 0;
    return dot / sqrt(na * nb);
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
```

```

cin >> n;
string line;
getline(cin, line);

vector<map<string, int>> docs(n + 1);
map<string, int> df;
for (int i = 1; i <= n; i++) {
    getline(cin, line);
    docs[i] = count_words(tokenize(line));
    for (auto [w, c] : docs[i]) df[w]++;
}

string query_line;
getline(cin, query_line);
map<string, int> query = count_words(tokenize(query_line));

int best_id = 1;
double best_score = -1;
for (int i = 1; i <= n; i++) {
    double score = tfidf_cosine(docs[i], query, df, n);
    if (score > best_score + 1e-12) {
        best_score = score;
        best_id = i;
    }
}

cout << fixed << setprecision(6) << best_id << ' ' << best_score << '\n';
return 0;
}

```

调用示例:

```

vector<string> words = tokenize("Apple, banana! apple");
auto cnt = count_words(words);

```

常见坑:

- 大小写是否敏感看题面; 本模板默认转小写。
- TF 是否除以文档长度看题面; 本模板用原始词频。
- IDF 公式版本很多, 以题面为准。
- 查询词不在任何文档中时, 本模板仍给它一个平滑 IDF。
- 相似度相同默认保留编号小的文档。

暴力/部分替代:

- 不会 TF-IDF: 先按共同词数量排序。
- 不会 cosine: 先按点积排序。
- 文本太大: 先只处理题目要求的关键词。

最小测试样例:

输入

```

2
apple banana
cat dog
apple

```

输出

```

1 0.707107

```

AI-06 聚类、归一化与线性回归

模块编号: AI-06

模块名称: k-means 聚类、特征归一化与线性回归路由

标签：AI、聚类、k-means、回归、归一化、梯度下降、C++17

一句话用途：当题目给无标签点集、聚类中心、迭代次数，或给线性回归训练规则时，用本模块按公式模拟。

题面触发词：

- 聚类、中心、最近中心、k-means。
- 归一化、标准化、min-max。
- 线性回归、均方误差、梯度下降。
- 学习率、迭代次数、权重更新。

什么时候用：

- 题目明确给 k、迭代次数和初始中心。
- 特征维度不高，点数中等。
- 线性回归给出更新公式或只要求预测。

不要什么时候用：

- 不要自行脑补随机初始化，题目通常会给初始中心或规则。
- 聚类结果 tie-break 必须看题面；本模板距离相同选编号小中心。
- 大规模矩阵运算不要写复杂库，按题面范围估算。

复杂度：

- k-means: $O(\text{iter} * n * k * d)$ 。
- 线性预测: $O(q * d)$ 。
- 梯度下降: $O(\text{epoch} * n * d)$ 。

依赖的标准容器：

- `vector<double>`: 点、中心、权重。
- `vector<int>`: 每个点所属中心。

输入如何整理：

```
int n, d, k, iter;
cin >> n >> d >> k >> iter;
vector<vector<double>> x(n + 1, vector<double>(d + 1));
```

接口：

`kmeans(points, n, d, k, iter) -> labels + centers.`

模板代码：

```
#include <bits/stdc++.h>
using namespace std;

double squared_distance(const vector<double> &a, const vector<double> &b, int d) {
    double res = 0;
    for (int i = 1; i <= d; i++) {
        double diff = a[i] - b[i];
        res += diff * diff;
    }
    return res;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, d, k, iter;
    cin >> n >> d >> k >> iter;
    vector<vector<double>> x(n + 1, vector<double>(d + 1));
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= d; j++) cin >> x[i][j];
    }

    vector<vector<double>> center(k + 1, vector<double>(d + 1));
    for (int c = 1; c <= k; c++) center[c] = x[c];
```

```

vector<int> label(n + 1, 1);
for (int it = 1; it <= iter; it++) {
    for (int i = 1; i <= n; i++) {
        int best = 1;
        double best_dist = squared_distance(x[i], center[1], d);
        for (int c = 2; c <= k; c++) {
            double cur = squared_distance(x[i], center[c], d);
            if (cur < best_dist - 1e-12) {
                best_dist = cur;
                best = c;
            }
        }
        label[i] = best;
    }
}

vector<vector<double>> sum(k + 1, vector<double>(d + 1, 0));
vector<int> cnt(k + 1, 0);
for (int i = 1; i <= n; i++) {
    int c = label[i];
    cnt[c]++;
    for (int j = 1; j <= d; j++) sum[c][j] += x[i][j];
}
for (int c = 1; c <= k; c++) {
    if (cnt[c] == 0) continue;
    for (int j = 1; j <= d; j++) center[c][j] = sum[c][j] / cnt[c];
}
}

cout << fixed << setprecision(6);
for (int i = 1; i <= n; i++) {
    if (i > 1) cout << ' ';
    cout << label[i];
}
cout << '\n';
for (int c = 1; c <= k; c++) {
    for (int j = 1; j <= d; j++) {
        if (j > 1) cout << ' ';
        cout << center[c][j];
    }
    cout << '\n';
}

return 0;
}

```

线性回归路由

预测公式:

$$\text{pred} = w[1]*x[1] + \dots + w[d]*x[d] + b$$

梯度下降常见更新:

```

err = pred - y
w[j] -= lr * err * x[j]
b -= lr * err

```

归一化路由

min-max: $x' = (x - \min) / (\max - \min)$

standard: $x' = (x - \text{mean}) / \text{std}$

常见坑:

- k-means 空簇怎么处理看题面; 本模板保持原中心。

- 距离相同 tie-break 选编号小中心。
- 初始中心很关键，通常取前 k 个点或题面给定。
- 回归训练要区分批量更新和在线更新。
- 标准差为 0 时要防御。

暴力/部分替代：

- k-means 不会迭代：只做一次最近中心分配。
- 回归不会训练：只做给定权重预测。
- 归一化不会：先用原特征跑距离。

最小测试样例：

输入

```
4 1 2 2
0
1
10
11
```

输出

```
1 1 2 2
0.500000
10.500000
```

AI-07 Markov 链、HMM 与 Viterbi

模块编号：AI-07

模块名称：Markov 状态转移、隐马尔可夫模型与 Viterbi 最可能路径

标签：AI、概率模型、Markov、HMM、Viterbi、动态规划、C++17

一句话用途：当题目出现状态转移概率、观测序列、最可能隐藏状态路径时，用 Viterbi 把概率模型变成 DP。

题面触发词：

- Markov、状态转移、转移矩阵。
- hidden state、observation、emission probability。
- 给观测序列，求最可能状态序列。
- 概率连乘、路径最大概率。

什么时候用：

- 下一步状态只依赖当前状态。
- 观测概率只依赖当前隐藏状态。
- 要最大概率路径，而不是总概率。

不要什么时候用：

- 只求所有路径概率总和时，用 forward DP，不是 Viterbi max。
- 概率极小，不能直接乘很多次；本模板用 log。
- 状态数和序列长很大时， $O(T \cdot n^2)$ 可能 TLE。

复杂度：

- Viterbi: $O(T \cdot n^2)$ 。
- 空间: $O(T \cdot n)$ 用于回溯路径；只求概率可滚动。

依赖的标准容器：

- `vector<vector<double>>`: log 概率 DP。
- `vector<vector<int>>`: 前驱路径。
- `vector<int>`: 观测序列。

输入如何整理：

```
int n, m, T;
cin >> n >> m >> T;
```

接口：

pi[i] 初始概率。
trans[i][j] 状态 i 到 j 的概率。
emit[i][o] 状态 i 产生观测 o 的概率。
obs[t] 第 t 个观测。

模板代码:

```
#include <bits/stdc++.h>
using namespace std;

const double NEG = -1e100;

double safe_log(double x) {
    if (x <= 0) return NEG;
    return log(x);
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m, T;
    cin >> n >> m >> T;
    if (n <= 0 || m <= 0 || T <= 0) throw runtime_error("bad size");

    vector<double> pi(n + 1);
    for (int i = 1; i <= n; i++) cin >> pi[i];

    vector<vector<double>> trans(n + 1, vector<double>(n + 1));
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) cin >> trans[i][j];
    }

    vector<vector<double>> emit(n + 1, vector<double>(m + 1));
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) cin >> emit[i][j];
    }

    vector<int> obs(T + 1);
    for (int t = 1; t <= T; t++) {
        cin >> obs[t];
        if (obs[t] < 1 || obs[t] > m) throw runtime_error("bad observation");
    }

    vector<vector<double>> dp(T + 1, vector<double>(n + 1, NEG));
    vector<vector<int>> pre(T + 1, vector<int>(n + 1, 0));

    for (int s = 1; s <= n; s++) {
        dp[1][s] = safe_log(pi[s]) + safe_log(emit[s][obs[1]]);
    }

    for (int t = 2; t <= T; t++) {
        for (int s = 1; s <= n; s++) {
            for (int p = 1; p <= n; p++) {
                double cur = dp[t - 1][p] + safe_log(trans[p][s]) + safe_log(emit[s][obs[t]]);
                if (cur > dp[t][s]) {
                    dp[t][s] = cur;
                    pre[t][s] = p;
                }
            }
        }
    }

    int last = 1;
```

```

for (int s = 2; s <= n; s++) {
    if (dp[T][s] > dp[T][last]) last = s;
}
if (dp[T][last] <= NEG / 2) {
    cout << fixed << setprecision(6) << 0.0 << '\n';
    cout << -1 << '\n';
    return 0;
}

vector<int> path(T + 1);
path[T] = last;
for (int t = T; t >= 2; t--) {
    if (pre[t][path[t]] == 0) throw runtime_error("broken path");
    path[t - 1] = pre[t][path[t]];
}

cout << fixed << setprecision(6) << exp(dp[T][last]) << '\n';
for (int t = 1; t <= T; t++) {
    if (t > 1) cout << ' ';
    cout << path[t];
}
cout << '\n';

return 0;
}

```

调用示例:

// 按 HMM 输入后输出最大概率和隐藏状态路径。

常见坑:

- 概率为 0 时 log 是负无穷, 要特殊处理。
- Viterbi 是取最大路径, 不是概率求和。
- 输出概率可能很小, 题面可能要求输出 log 概率。
- 如果路径 tie-break 有要求, 要在 `cur > dp` 改成带编号比较。

暴力/部分替代:

- 状态数小、T 小: 暴力枚举所有路径。
- 不会 log: 小数据直接乘概率。
- 只要最终状态: 不保存 pre, 滚动数组。

最小测试样例:

输入

```

2 2 3
0.5 0.5
0.9 0.1
0.2 0.8
0.8 0.2
0.1 0.9
1 2 2

```

输出

```

0.025920
1 2 2

```

AI-08 神经网络前向传播、激活函数与 Softmax

模块编号: AI-08

模块名称: 神经网络基础前向传播与分类输出

标签: AI、神经网络、前向传播、ReLU、Sigmoid、Softmax、分类、C++17

一句话用途：当题目给权重、偏置、输入向量和激活函数，要求你算模型输出或预测类别时，用本模块按矩阵向量公式模拟前向传播。

题面触发词：

- 神经元、权重、偏置、激活函数。
- ReLU、Sigmoid、Softmax。
- logits、概率、预测类别。
- 前向传播、推理、模型参数。

什么时候用：

- 题目只要求推理，不要求复杂训练。
- 网络层数少，权重直接给出。
- 分类输出按最大概率或最大 logit。

不要什么时候用：

- 不要实现反向传播，除非题面给非常明确的更新公式。
- 不要使用第三方矩阵库。
- 大矩阵乘法仍按 $O(\text{层数} * \text{输出维度} * \text{输入维度})$ 估算。

复杂度：

- 单层全连接： $O(c*d)$ 。
- 多层：各层 输出维度 * 输入维度 求和。
- softmax： $O(c)$ 。

依赖的标准容器：

- `vector<double>`：向量、权重、偏置。
- `vector<vector<double>>`：权重矩阵，1-index。

输入如何整理：

```
string act;
int c, d;
cin >> act >> c >> d;
```

接口：

```
z[i] = bias[i] + sum_j w[i][j] * x[j]
activation: none / relu / sigmoid / softmax
prediction = argmax output[i], tie 选编号小。
```

模板代码：

```
#include <bits/stdc++.h>
using namespace std;

double sigmoid(double x) {
    if (x >= 0) {
        double e = exp(-x);
        return 1.0 / (1.0 + e);
    }
    double e = exp(x);
    return e / (1.0 + e);
}

vector<double> softmax(vector<double> z) {
    int n = (int)z.size() - 1;
    double mx = z[1];
    for (int i = 2; i <= n; i++) mx = max(mx, z[i]);
    double sum = 0;
    for (int i = 1; i <= n; i++) {
        z[i] = exp(z[i] - mx);
        sum += z[i];
    }
    for (int i = 1; i <= n; i++) z[i] /= sum;
    return z;
}
```



```

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string act;
    int c, d;
    cin >> act >> c >> d;

    vector<double> x(d + 1);
    for (int j = 1; j <= d; j++) cin >> x[j];

    vector<vector<double>> w(c + 1, vector<double>(d + 1));
    vector<double> b(c + 1);
    for (int i = 1; i <= c; i++) {
        for (int j = 1; j <= d; j++) cin >> w[i][j];
        cin >> b[i];
    }

    vector<double> y(c + 1, 0);
    for (int i = 1; i <= c; i++) {
        y[i] = b[i];
        for (int j = 1; j <= d; j++) y[i] += w[i][j] * x[j];
    }

    if (act == "relu") {
        for (int i = 1; i <= c; i++) y[i] = max(0.0, y[i]);
    } else if (act == "sigmoid") {
        for (int i = 1; i <= c; i++) y[i] = sigmoid(y[i]);
    } else if (act == "softmax") {
        y = softmax(y);
    }

    int pred = 1;
    for (int i = 2; i <= c; i++) {
        if (y[i] > y[pred] + 1e-12) pred = i;
    }

    cout << fixed << setprecision(6);
    cout << pred << '\n';
    for (int i = 1; i <= c; i++) {
        if (i > 1) cout << ' ';
        cout << y[i];
    }
    cout << '\n';

    return 0;
}

```

调用示例:

// softmax 输出概率, argmax 为预测类别。

常见坑:

- softmax 要减最大 logit, 防止 exp 溢出。
- 类别编号通常 1-index, 输出 tie-break 选编号小。
- sigmoid 大负数直接 $\exp(-x)$ 会溢出, 本模板分情况写。
- ReLU 会把负数变成 0。
- 题面可能要求输出 logits 而不是概率, 仔细看输出格式。

暴力/部分替代:

- 多层不会写: 先写单层全连接。
- softmax 不会写: 先输出最大 logit 的类别。
- 训练不会写: 先实现前向推理。

最小测试样例:

输入

```
softmax 2 2
1 2
1 0 0
0 1 0
```

输出

```
2
0.268941 0.731059
```

AI-09 强化学习、MDP 与值迭代

模块编号: AI-09

模块名称: 状态、动作、奖励、策略与 Value Iteration

标签: AI、强化学习、MDP、状态、动作、奖励、策略、值迭代、C++17

一句话用途: 当题目出现状态、动作、奖励、折扣因子、最优策略时, 把它看成图上的 DP: $V[s] = \max_a(\text{reward} + \gamma * V[\text{next}])$ 。

题面触发词:

- 状态、动作、奖励、策略。
- 折扣因子 γ 。
- value、Q value、value iteration。
- grid world、智能体每步获得 reward。

什么时候用:

- 状态和动作数量明确, 转移规则给定。
- 题目要求迭代固定轮数或直到收敛。
- 转移是确定性的或概率转移可枚举。

不要什么时候用:

- 不要把 RL 想复杂; 机考大概率是按公式模拟。
- 如果是普通最短路/最长路, 直接图算法更清晰。
- γ 接近 1 且有正环时, 值可能不断变大, 题面应给迭代次数或终止状态。

复杂度:

- 确定性 value iteration: $O(\text{iter} * \text{states} * \text{actions})$ 。
- 概率转移: $O(\text{iter} * \text{states} * \text{actions} * \text{next_states})$ 。

依赖的标准容器:

- `vector<vector<int>> next_state`
- `vector<vector<double>> reward`
- `vector<double> V`

输入如何整理:

```
int n, m, iter;
double gamma;
cin >> n >> m >> iter >> gamma;
```

接口:

`next_state[s][a]`: 状态 s 执行动作 a 后到达的状态。

`reward[s][a]`: 该动作即时奖励。

`V[s]`: 状态价值。

模板代码:

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
```

```

cin.tie(nullptr);

int n, m, iter;
double gamma;
cin >> n >> m >> iter >> gamma;

vector<vector<int>> nxt(n + 1, vector<int>(m + 1));
vector<vector<double>> reward(n + 1, vector<double>(m + 1));
for (int s = 1; s <= n; s++) {
    for (int a = 1; a <= m; a++) {
        cin >> nxt[s][a] >> reward[s][a];
    }
}

vector<double> V(n + 1, 0);
for (int it = 1; it <= iter; it++) {
    vector<double> nV(n + 1, 0);
    for (int s = 1; s <= n; s++) {
        nV[s] = reward[s][1] + gamma * V[nxt[s][1]];
        for (int a = 2; a <= m; a++) {
            nV[s] = max(nV[s], reward[s][a] + gamma * V[nxt[s][a]]);
        }
    }
    V = nV;
}

cout << fixed << setprecision(6);
for (int s = 1; s <= n; s++) {
    if (s > 1) cout << ' ';
    cout << V[s];
}
cout << '\n';

return 0;
}

```

Q-learning 路由

如果题面给经验序列 $(s, a, r, next)$ 和学习率 α :

$$Q[s][a] = Q[s][a] + \alpha * (r + \gamma * \max Q[next][*] - Q[s][a])$$

这就是按公式模拟，不需要理解复杂理论。

调用示例:

// 固定 *iter* 轮 *value iteration* 后输出 $V[1..n]$ 。

常见坑:

- value iteration 每一轮要用上一轮 V 更新新 V ，不要原地更新，除非题目要求。
- γ 是 double，输出精度按题面。
- 状态编号、动作编号按题面，资料默认 1-index。
- 终止状态通常所有动作回到自己、奖励 0，或题面单独说明。
- Q-learning 是在线更新，value iteration 是整轮同步更新。

暴力/部分分替代:

- 不会 value iteration: 先模拟给定策略，不取 $\max$ 。
- 不会概率转移: 先处理确定性转移子任务。
- 不会收敛判断: 按固定迭代次数输出。

最小测试样例:

输入

3 2 3 1.0

2 0 3 1

3 5 1 0

3 0 3 0

输出

5.000000 5.000000 0.000000

AI-10 Special Judge、评分指标与模型选择策略

模块编号: AI-10

模块名称: AI 题 Special Judge 评分、指标计算与 baseline 选择

标签: AI、Special Judge、准确率、误差、F1、模型选择、C++17

一句话用途: 当题目不是唯一答案, 而是按准确率、误差、相似度、得分函数评测时, 用本模块先写一个确定性 baseline, 再按数据范围逐步升级。

题面触发词:

- Special Judge、得分、score、accuracy、loss、error。
- 预测值、真实值、训练集、验证集、测试集。
- 平均误差、均方误差、RMSE、F1、precision、recall。
- 输出任意满足条件的模型结果、越接近越高分。

什么时候用:

- 输出可以不是唯一标准答案。
- 题目给训练数据和隐藏测试, 要求你输出预测、分类或参数。
- 可以多次提交, 且最终取最高分, 适合 baseline -> 调参 -> 升级模型。

不要什么时候用:

- 如果题目是普通 OJ 精确答案, 不要把它当机器学习题。
- 不要随机输出不可复现结果; 考场调参必须保证同一代码每次一样。
- 不要为了复杂模型牺牲合法输出和部分分 baseline。

复杂度:

- 指标计算: $O(n)$ 。
- 选择多数类/平均值 baseline: $O(n)$ 。
- 网格调参: $O(\text{参数组合数} * \text{验证集规模} * \text{单次预测复杂度})$ 。

依赖的标准容器:

- `vector<double>`: 真实值、预测值。
- `vector<int>`: 真实类别、预测类别。
- `map<int,int>`: 多数类统计。

输入如何整理:

```
int n;
cin >> n;
vector<double> y(n + 1), pred(n + 1);
for (int i = 1; i <= n; i++) cin >> y[i] >> pred[i];
```

接口:

metric mode:

accuracy: 输入 n 行真实类别 预测类别, 输出准确率。

mae: 输入 n 行真实值 预测值, 输出平均绝对误差。

rmse: 输入 n 行真实值 预测值, 输出均方根误差。

f1: 输入 n 行真实二分类标签 预测二分类标签, 正类固定为 1。

模板代码:

```
#include <bits/stdc++.h>
using namespace std;

double metric_accuracy(const vector<int> &real_label, const vector<int> &pred_label, int n) {
    int correct = 0;
    for (int i = 1; i <= n; i++) {
        if (real_label[i] == pred_label[i]) correct++;
    }
    return correct / n;
}
```

```

    }
    return n == 0 ? 0.0 : (double)correct / n;
}

double metric_mae(const vector<double> &y, const vector<double> &pred, int n) {
    double sum = 0;
    for (int i = 1; i <= n; i++) sum += fabs(y[i] - pred[i]);
    return n == 0 ? 0.0 : sum / n;
}

double metric_rmse(const vector<double> &y, const vector<double> &pred, int n) {
    double sum = 0;
    for (int i = 1; i <= n; i++) {
        double e = y[i] - pred[i];
        sum += e * e;
    }
    return n == 0 ? 0.0 : sqrt(sum / n);
}

double metric_binary_f1(const vector<int> &real_label, const vector<int> &pred_label, int n) {
    int tp = 0, fp = 0, fn = 0;
    for (int i = 1; i <= n; i++) {
        if (real_label[i] == 1 && pred_label[i] == 1) tp++;
        if (real_label[i] != 1 && pred_label[i] == 1) fp++;
        if (real_label[i] == 1 && pred_label[i] != 1) fn++;
    }
    double precision = (tp + fp == 0) ? 0.0 : (double)tp / (tp + fp);
    double recall = (tp + fn == 0) ? 0.0 : (double)tp / (tp + fn);
    if (precision + recall == 0) return 0.0;
    return 2.0 * precision * recall / (precision + recall);
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    string mode;
    int n;
    cin >> mode >> n;

    cout << fixed << setprecision(6);
    if (mode == "accuracy" || mode == "f1") {
        vector<int> real_label(n + 1), pred_label(n + 1);
        for (int i = 1; i <= n; i++) cin >> real_label[i] >> pred_label[i];
        if (mode == "accuracy") cout << metric_accuracy(real_label, pred_label, n) << '\n';
        else cout << metric_binary_f1(real_label, pred_label, n) << '\n';
    } else {
        vector<double> y(n + 1), pred(n + 1);
        for (int i = 1; i <= n; i++) cin >> y[i] >> pred[i];
        if (mode == "mae") cout << metric_mae(y, pred, n) << '\n';
        else if (mode == "rmse") cout << metric_rmse(y, pred, n) << '\n';
    }

    return 0;
}

```

SPJ 考场 baseline 路由

输出类型	第一个可提交 baseline	升级方向
分类标签	训练集多数类	AI-02 kNN/朴素贝叶斯, AI-11 SVM
连续数值	训练集平均值/中位数	AI-13 线性回归, AI-06 归一化
排名/推荐	全局热门度	AI-03/05 相似度、Top-K、TF-IDF

输出类型	第一个可提交 baseline	升级方向
聚类编号	按输入顺序分组	AI-06 k-means、距离阈值
路径/动作	合法最短/贪心动作	GRAPH-02/03, AI-01 A*
小网络预测	线性模型/前向传播	AI-12 多层前向, AI-14 小网络反传
计算图梯度	数值差分	AI-15 反向模式自动求导

验证集与调参策略

1. 先把训练集前 80% 当 train, 后 20% 当 valid。
2. 写最简单 baseline, 算 valid 分数。
3. 只调 1-2 个参数: k、学习率、迭代轮数、距离权重。
4. 固定随机性: 不要 rand; 如果必须打散, 用固定种子。
5. 最后用全部训练集重训一次, 再输出测试集预测。

常见坑:

- Special Judge 仍然会检查输出格式, 格式错就是 0 分。
- 评分指标越大越好还是越小越好必须确认。
- 浮点输出一般多打几位, `fixed << setprecision(10)` 很稳。
- hidden test 分布可能和样例不同, 不要只为样例调参。
- 如果类标不是 1..c, 先离散化或用 map 统计。

暴力/部分分替代:

- 完全不会模型: 分类输出多数类, 回归输出均值, 推荐输出热门项。
- 数据小: 直接 kNN 或枚举参数。
- 不会训练: 按题面给定权重做推理, 或输出合法默认值。

最小测试样例:

输入

```
f1 5
1 1
1 0
0 1
0 0
1 1
```

输出

```
0.666667
```

AI-11 线性 SVM、间隔分类与 Hinge Loss

模块编号: AI-11

模块名称: 线性 SVM 训练与预测模板

标签: AI、监督学习、SVM、线性分类、Hinge Loss、SGD、C++17

一句话用途: 当题目给二分类样本、学习率、正则系数和训练轮数, 要求按线性 SVM 或最大间隔思想预测时, 用本模块直接模拟更新。

题面触发词:

- SVM、support vector、margin、hinge loss。
- 二分类、标签 -1/+1。
- 学习率、lambda、正则化、迭代轮数。
- $\max(0, 1 - y*(w*x+b))$ 。

什么时候用:

- 题目明确要求线性 SVM 或给出 hinge loss 更新规则。
- 特征维度不高, 训练轮数有限。
- 输出类别只需要正负类预测。

不要什么时候用:

- 不要实现核函数 SVM, 除非题面给非常具体的公式。

- 标签不是 -1/+1 时要先转换，不要直接拿 0/1 进更新。
- 如果题目只是普通线性分类，感知机可能更短；SVM 是更稳的备用模板。

复杂度：

- 训练： $O(\text{epoch} * n * d)$ 。
- 单个预测： $O(d)$ 。

依赖的标准容器：

- `vector<vector<double>>`: 1-index 特征矩阵。
- `vector<int>`: 标签。
- `vector<double>`: 权重。

输入如何整理：

```
int n, d, epoch;
double lr, lambda;
cin >> n >> d >> epoch >> lr >> lambda;
```

接口：

```
score = w[1]*x[1] + ... + w[d]*x[d] + b
pred = score >= 0 ? +1 : -1
if y * score < 1:
    w[j] = w[j] - lr * lambda * w[j] + lr * y * x[j]
    b = b + lr * y
else:
    w[j] = w[j] - lr * lambda * w[j]
```

模板代码：

```
#include <bits/stdc++.h>
using namespace std;

double dot_product(const vector<double> &w, const vector<double> &x, int d) {
    double res = 0;
    for (int j = 1; j <= d; j++) res += w[j] * x[j];
    return res;
}

int predict_svm(const vector<double> &w, double b, const vector<double> &x, int d) {
    double score = dot_product(w, x, d) + b;
    return score >= 0 ? 1 : -1;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, d, epoch;
    double lr, lambda;
    cin >> n >> d >> epoch >> lr >> lambda;

    vector<vector<double>> x(n + 1, vector<double>(d + 1));
    vector<int> y(n + 1);
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= d; j++) cin >> x[i][j];
        cin >> y[i];
        if (y[i] == 0) y[i] = -1;
    }

    vector<double> w(d + 1, 0);
    double b = 0;

    for (int ep = 1; ep <= epoch; ep++) {
        for (int i = 1; i <= n; i++) {
            double score = dot_product(w, x[i], d) + b;
```

```

        double margin = y[i] * score;
        for (int j = 1; j <= d; j++) {
            w[j] -= lr * lambda * w[j];
        }
        if (margin < 1.0) {
            for (int j = 1; j <= d; j++) {
                w[j] += lr * y[i] * x[i][j];
            }
            b += lr * y[i];
        }
    }
}

int q;
cin >> q;
for (int qi = 1; qi <= q; qi++) {
    vector<double> query(d + 1);
    for (int j = 1; j <= d; j++) cin >> query[j];
    cout << predict_svm(w, b, query, d) << '\n';
}

return 0;
}

```

调用示例:

// 标签输入若是 0/1, 模板会把 0 转成 -1。

常见坑:

- score == 0 的 tie-break 本模板预测 +1, 题面不同就改。
- 有正则项时, 即使分类正确也要衰减 w。
- SVM 的 lambda、lr、epoch 常数很影响 SPJ 分数, 优先按验证集调。
- 特征尺度差很多时, 先做归一化, 否则训练容易被某一维支配。

暴力/部分分替代:

- 不会 SVM: 先写感知机; 只在错误时更新。
- 不会训练: 输出多数类或按给定权重预测。
- 数据很小: 用 kNN 通常更稳。

最小测试样例:

输入
2 1 1 1 0
1 1
-1 -1
3
2
-2
0

输出

1
-1
1

AI-12 多层 DNN 前向传播模板

模块编号: AI-12

模块名称: 多层全连接神经网络前向传播

标签: AI、DNN、神经网络、全连接层、ReLU、Sigmoid、Softmax、C++17

一句话用途: 当题目给多层权重、偏置、激活函数, 要求算输出概率或分类结果时, 用本模块按层模拟前向传播。

题面触发词：

- 多层感知机、DNN、MLP。
- layer、weight、bias、activation。
- ReLU、Sigmoid、Tanh、Softmax。
- 前向传播、推理、概率输出。

什么时候用：

- 题目只要求 forward，不要求反向传播。
- 每层规模不大，可以用 vector 或静态数组模拟矩阵向量乘。
- 输出要求类别、logit 或概率。

不要什么时候用：

- 不要手写复杂训练框架。
- 不要把 softmax 放在中间层，除非题面这样要求。
- 大矩阵要估算 $\text{sum}(\text{in_dim} * \text{out_dim})$ ，避免超时。

复杂度：

- 总复杂度：所有层 $O(\text{in_dim} * \text{out_dim})$ 求和。
- 空间：当前层向量 + 下一层向量即可。

依赖的标准容器：

- `vector<double>`：当前向量、下一层向量。
- `vector<vector<double>>`：当前层权重。

输入如何整理：

```
int layer_count;
cin >> layer_count;
int dim;
cin >> dim;
vector<double> cur(dim + 1);
```

接口：

每层输入：

out_dim activation

out_dim 行，每行 in_dim 个权重，最后一个数是 bias

activation: none / relu / sigmoid / tanh / softmax

模板代码：

```
#include <bits/stdc++.h>
using namespace std;

double sigmoid(double x) {
    if (x >= 0) {
        double e = exp(-x);
        return 1.0 / (1.0 + e);
    }
    double e = exp(x);
    return e / (1.0 + e);
}

void apply_activation(vector<double> &a, const string &act) {
    int n = (int)a.size() - 1;
    if (act == "relu") {
        for (int i = 1; i <= n; i++) a[i] = max(0.0, a[i]);
    } else if (act == "sigmoid") {
        for (int i = 1; i <= n; i++) a[i] = sigmoid(a[i]);
    } else if (act == "tanh") {
        for (int i = 1; i <= n; i++) a[i] = tanh(a[i]);
    } else if (act == "softmax") {
        double mx = a[1];
        for (int i = 2; i <= n; i++) mx = max(mx, a[i]);
        double sum = 0;
```

```

        for (int i = 1; i <= n; i++) {
            a[i] = exp(a[i] - mx);
            sum += a[i];
        }
        for (int i = 1; i <= n; i++) a[i] /= sum;
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int layer_count;
    cin >> layer_count;

    int dim;
    cin >> dim;
    vector<double> cur(dim + 1);
    for (int j = 1; j <= dim; j++) cin >> cur[j];

    for (int layer = 1; layer <= layer_count; layer++) {
        int out_dim;
        string act;
        cin >> out_dim >> act;

        vector<double> nxt(out_dim + 1, 0);
        for (int i = 1; i <= out_dim; i++) {
            for (int j = 1; j <= dim; j++) {
                double w;
                cin >> w;
                nxt[i] += w * cur[j];
            }
            double b;
            cin >> b;
            nxt[i] += b;
        }

        apply_activation(nxt, act);
        cur = nxt;
        dim = out_dim;
    }

    int pred = 1;
    for (int i = 2; i <= dim; i++) {
        if (cur[i] > cur[pred] + 1e-12) pred = i;
    }

    cout << fixed << setprecision(6);
    cout << pred << '\n';
    for (int i = 1; i <= dim; i++) {
        if (i > 1) cout << ' ';
        cout << cur[i];
    }
    cout << '\n';

    return 0;
}

```

调用示例:

// 每层只保留当前向量, 适合手写多层前向传播题。

常见坑:

- softmax 要减最大值, 防止 exp 溢出。

- 权重矩阵方向看题面：本模板是 out_dim 行、in_dim 列。
- 输出类别 tie-break 本模板选编号小。
- 题面如果输出 logit，就不要对最后一层 softmax。

暴力/部分替代：

- 多层太长：先支持 none/relu/softmax 三种激活。
- 不会 sigmoid/tanh：用库函数 exp/tanh，注意精度。
- 训练不会：只做前向推理，也常能拿对应子任务分。

最小测试样例：

输入

```
2
2
1 2
2 relu
1 0 0
0 1 0
2 softmax
1 0 0
0 1 0
```

输出

```
2
0.268941 0.731059
```

AI-13 线性回归与梯度下降训练

模块编号：AI-13

模块名称：线性回归在线梯度下降与预测

标签：AI、监督学习、回归、线性回归、梯度下降、MSE、C++17

一句话用途：当题目给连续标签、学习率和训练轮数，要求按线性回归公式训练并预测时，用本模块写一个可调的回归 baseline。

题面触发词：

- regression、linear model、MSE、loss。
- 连续值预测、房价、评分、概率分数。
- 学习率、epoch、gradient descent。
- 权重、偏置、预测误差。

什么时候用：

- 输出是连续值，Special Judge 按误差给分。
- 特征维度不高，可以手写线性模型。
- 题面直接给更新公式或允许自选简单模型。

不要什么时候用：

- 不要把所有回归题都强行套线性模型；先看是否有明显公式。
- 特征量级差异很大时，最好先归一化。
- 学习率过大会发散，SPJ 题要小步调参。

复杂度：

- 训练： $O(\text{epoch} * n * d)$ 。
- 单次预测： $O(d)$ 。

依赖的标准容器：

- `vector<vector<double>>`：训练特征。
- `vector<double>`：标签、权重、查询。

输入如何整理：

```
int n, d, epoch;
double lr;
cin >> n >> d >> epoch >> lr;
```

接口:

```
pred = w[1]*x[1] + ... + w[d]*x[d] + b
err = pred - y
w[j] -= lr * err * x[j]
b -= lr * err
```

模板代码:

```
#include <bits/stdc++.h>
using namespace std;

double predict_linear(const vector<double> &w, double b, const vector<double> &x, int d) {
    double res = b;
    for (int j = 1; j <= d; j++) res += w[j] * x[j];
    return res;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, d, epoch;
    double lr;
    cin >> n >> d >> epoch >> lr;

    vector<vector<double>> x(n + 1, vector<double>(d + 1));
    vector<double> y(n + 1);
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= d; j++) cin >> x[i][j];
        cin >> y[i];
    }

    vector<double> w(d + 1, 0);
    double b = 0;

    for (int ep = 1; ep <= epoch; ep++) {
        for (int i = 1; i <= n; i++) {
            double pred = predict_linear(w, b, x[i], d);
            double err = pred - y[i];
            for (int j = 1; j <= d; j++) {
                w[j] -= lr * err * x[i][j];
            }
            b -= lr * err;
        }
    }

    int q;
    cin >> q;
    cout << fixed << setprecision(6);
    for (int qi = 1; qi <= q; qi++) {
        vector<double> query(d + 1);
        for (int j = 1; j <= d; j++) cin >> query[j];
        cout << predict_linear(w, b, query, d) << '\n';
    }

    return 0;
}
```

SPJ 调参建议

现象	调整
预测值爆炸	降低 lr, 先做归一化
训练太慢	减少 epoch 或只用部分特征
valid 分数差	尝试 lr = 0.001/0.01/0.05/0.1
输出范围有限	最后 pred = min(max(pred, L), R)

常见坑:

- 在线更新和批量更新结果不同, 题面若规定必须照题面。
- 若用 MSE 的完整梯度, 有些写法会多一个 2, 这可合并到学习率里。
- 回归输出通常多打几位小数。
- 标签和特征很大时, double 比 float 稳。

暴力/部分分替代:

- 不会训练: 输出训练标签平均值。
- 特征只有一维: 先尝试按比例或最小二乘公式。
- SPJ 有多次提交: 先均值 baseline, 再上线性回归, 再调学习率。

最小测试样例:

输入

```
1 1 1 1
1 2
1
3
```

输出

```
8.000000
```

AI-14 反向传播、链式法则与小网络训练

模块编号: AI-14

模块名称: 二层神经网络反向传播模板

标签: AI、反向传播、链式法则、神经网络训练、梯度下降、C++17

一句话用途: 当题目给一个小神经网络、损失函数和学习率, 要求你手算或模拟一轮/多轮参数更新时, 用本模块按链式法则从输出层往前传梯度。

题面触发词:

- backward、backpropagation、gradient、chain rule。
- loss、MSE、cross entropy。
- 学习率、参数更新、一轮训练。
- 隐藏层、输出层、ReLU、sigmoid。

什么时候用:

- 网络很小, 题目要求按公式模拟训练。
- 题面给初始权重、偏置、学习率和训练轮数。
- Special Judge 按预测误差打分, 你想在线性模型之外再尝试小网络。

不要什么时候用:

- 不要现场写通用深度学习框架。
- 不要在数据很大时盲目训练多层网络, 容易 TLE 且调参困难。
- 如果题目只要前向传播, 翻 AI-08/12, 不要写反向传播。

复杂度:

- 本模板二层网络每个样本: $O(d \cdot h)$ 。
- 总训练: $O(\text{epoch} \cdot n \cdot d \cdot h)$ 。

依赖的标准容器:

- `vector<vector<double>>`: 输入层到隐藏层权重。
- `vector<double>`: 隐藏层、输出层权重和偏置。

输入如何整理:

```
int n, d, h, epoch;
double lr;
cin >> n >> d >> h >> epoch >> lr;
```

接口:

隐藏层:

```
z1[j] = b1[j] + sum_k W1[j][k] * x[k]
a1[j] = relu(z1[j])
```

输出层:

```
z2 = b2 + sum_j W2[j] * a1[j]
yhat = sigmoid(z2)
loss = 0.5 * (yhat - y)^2
```

反向传播:

```
dz2 = (yhat - y) * yhat * (1 - yhat)
dW2[j] = dz2 * a1[j]
da1[j] = dz2 * W2[j]
dz1[j] = da1[j] * (z1[j] > 0)
dW1[j][k] = dz1[j] * x[k]
```

多分类 softmax + 交叉熵规则

很多模拟题会把输出层写成多分类 softmax, 此时最常见、也最好背的是下面这一句:

```
p = softmax(z)
loss = -log(p[label])
输出层梯度: dz[k] = p[k] - (k == label)
```

如果隐藏层是 sigmoid:

```
delta1[i] = a1[i] * (1 - a1[i]) * sum_k W2[k][i] * dz[k]
```

如果隐藏层是 ReLU:

```
delta1[i] = (z1[i] > 0 ? 1 : 0) * sum_k W2[k][i] * dz[k]
```

考场判断:

二分类 + 单输出概率 -> 可以用本模块完整代码。

多分类 + softmax -> 用本节公式, 把输出层从一个数改成 c 个数。

题面只问梯度, 不问训练 -> 也可以翻 AI-15 计算图自动求导。

模板代码:

```
#include <bits/stdc++.h>
using namespace std;

double sigmoid(double x) {
    if (x >= 0) {
        double e = exp(-x);
        return 1.0 / (1.0 + e);
    }
    double e = exp(x);
    return e / (1.0 + e);
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, d, h, epoch;
    double lr;
    cin >> n >> d >> h >> epoch >> lr;

    vector<vector<double>> x(n + 1, vector<double>(d + 1));
```

```

vector<double> target(n + 1);
for (int i = 1; i <= n; i++) {
    for (int k = 1; k <= d; k++) cin >> x[i][k];
    cin >> target[i];
}

vector<vector<double>> w1(h + 1, vector<double>(d + 1));
vector<double> b1(h + 1);
for (int j = 1; j <= h; j++) {
    for (int k = 1; k <= d; k++) cin >> w1[j][k];
    cin >> b1[j];
}

vector<double> w2(h + 1);
double b2;
for (int j = 1; j <= h; j++) cin >> w2[j];
cin >> b2;

for (int ep = 1; ep <= epoch; ep++) {
    for (int i = 1; i <= n; i++) {
        vector<double> z1(h + 1), a1(h + 1);
        for (int j = 1; j <= h; j++) {
            z1[j] = b1[j];
            for (int k = 1; k <= d; k++) z1[j] += w1[j][k] * x[i][k];
            a1[j] = max(0.0, z1[j]);
        }

        double z2 = b2;
        for (int j = 1; j <= h; j++) z2 += w2[j] * a1[j];
        double yhat = sigmoid(z2);

        double dz2 = (yhat - target[i]) * yhat * (1.0 - yhat);
        vector<double> old_w2 = w2;

        for (int j = 1; j <= h; j++) {
            double grad_w2 = dz2 * a1[j];
            w2[j] -= lr * grad_w2;
        }
        b2 -= lr * dz2;

        for (int j = 1; j <= h; j++) {
            double da1 = dz2 * old_w2[j];
            double dz1 = (z1[j] > 0.0) ? da1 : 0.0;
            for (int k = 1; k <= d; k++) {
                double grad_w1 = dz1 * x[i][k];
                w1[j][k] -= lr * grad_w1;
            }
            b1[j] -= lr * dz1;
        }
    }
}

int q;
cin >> q;
cout << fixed << setprecision(6);
for (int qi = 1; qi <= q; qi++) {
    vector<double> query(d + 1);
    for (int k = 1; k <= d; k++) cin >> query[k];

    vector<double> a1(h + 1);
    for (int j = 1; j <= h; j++) {
        double z = b1[j];
        for (int k = 1; k <= d; k++) z += w1[j][k] * query[k];
    }
}

```

```

        a1[j] = max(0.0, z);
    }

    double z2 = b2;
    for (int j = 1; j <= h; j++) z2 += w2[j] * a1[j];
    cout << sigmoid(z2) << '\n';
}

return 0;
}

```

调用示例:

// 先用 `epoch=1` 对样例复现; 确认方向正确后再调 `Lr` 和 `epoch`。

常见坑:

- 更新隐藏层时必须用更新前的 `w2` 计算 `da1`, 本模板用 `old_w2` 保存。
- ReLU 在 $z \leq 0$ 时梯度为 0; $z == 0$ 题面若有规定按题面。
- MSE + sigmoid 的 `dz2` 比交叉熵多乘 `yhat*(1-yhat)`。
- 学习率太大会发散, SPJ 题先用小学习率。
- 反向传播本质是链式法则, 不要试图背一堆公式; 从输出层往前乘局部导数。

暴力/部分替代:

- 不会训练: 只做前向传播, 翻 AI-08/12。
- 不会隐藏层: 先写线性回归或 SVM, 翻 AI-11/13。
- SPJ 有多次提交: 先均值/多数类 baseline, 再小网络调参。

最小测试样例:

输入

```

1 1 1 1 1
1 1
0 0
0 0
1
1

```

输出

```

0.531209

```

AI-15 计算图与反向模式自动求梯度

模块编号: AI-15

模块名称: 表达式计算图的自动微分模板

标签: AI、自动求导、计算图、反向模式、梯度、链式法则、C++17

一句话用途: 当题目给一串计算图节点, 要求输出表达式值和各变量梯度时, 用本模块按拓扑顺序前向算值、逆序反向传梯度。

题面触发词:

- computational graph、autograd、automatic differentiation。
- 变量、常量、节点、边、操作符。
- 求偏导、gradient、backward。
- 链式法则、反向传播。

什么时候用:

- 计算图是有向无环图, 节点编号按依赖顺序给出。
- 需要对一个输出节点求所有输入变量梯度。
- 操作只有 `+` `-` `*` `/` `sin` `cos` `exp` `log` `relu` 等常见函数。

不要什么时候用:

- 图中有环时不能直接用本模板。
- 多输出函数要看题面, 是对哪个输出反传, 还是多个输出梯度相加。

- $\log(x)$ 要求 $x > 0$, 除法要求分母非 0。

复杂度:

- 前向计算: $O(\text{节点数})$ 。
- 反向传播: $O(\text{节点数})$ 。

依赖的标准容器:

- `vector<Node>`: 1-index 存节点。
- `vector<int>`: 变量节点编号。

输入如何整理:

```
int n;
cin >> n;
// 第 i 行描述第 i 个节点, 依赖编号必须小于 i。
```

接口:

支持节点:

```
var value
const value
add a b
sub a b
mul a b
div a b
sin a
cos a
exp a
log a
relu a
```

默认输出节点是 `n`。

输出: `value_of_node_n`, 然后按变量出现顺序输出梯度。

也可以封装成 AD 类

如果题目不是逐行给计算图, 而是要求你在代码里动态拼公式, 可以把每个操作封装成函数:

```
int x = var(2.0)
int y = var(3.0)
int z = add(mul(x, y), sin_node(x))
backward(z)
grad(x), grad(y)
```

类封装和下面的逐行解析模板本质相同: 每创建一个新节点, 就把操作类型、左右儿子、当前值存进数组; 最后从 `root` 逆序累加梯度。

模板代码:

```
#include <bits/stdc++.h>
using namespace std;

struct Node {
    string op;
    int l = 0, r = 0;
    double val = 0;
    double grad = 0;
    bool is_var = false;
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;
    vector<Node> a(n + 1);
    vector<int> vars;
```

```

auto check_child = [&](int id, int cur) {
    if (id <= 0 || id >= cur) throw runtime_error("bad node reference");
};

for (int i = 1; i <= n; i++) {
    cin >> a[i].op;
    if (a[i].op == "var" || a[i].op == "const") {
        cin >> a[i].val;
        if (a[i].op == "var") {
            a[i].is_var = true;
            vars.push_back(i);
        }
    } else if (a[i].op == "add" || a[i].op == "sub" ||
                a[i].op == "mul" || a[i].op == "div") {
        cin >> a[i].l >> a[i].r;
        int l = a[i].l, r = a[i].r;
        check_child(l, i);
        check_child(r, i);
        if (a[i].op == "add") a[i].val = a[l].val + a[r].val;
        if (a[i].op == "sub") a[i].val = a[l].val - a[r].val;
        if (a[i].op == "mul") a[i].val = a[l].val * a[r].val;
        if (a[i].op == "div") {
            if (fabs(a[r].val) < 1e-15) throw runtime_error("division by zero");
            a[i].val = a[l].val / a[r].val;
        }
    } else if (a[i].op == "sin" || a[i].op == "cos" ||
                a[i].op == "exp" || a[i].op == "log" ||
                a[i].op == "relu") {
        cin >> a[i].l;
        int l = a[i].l;
        check_child(l, i);
        if (a[i].op == "sin") a[i].val = sin(a[l].val);
        if (a[i].op == "cos") a[i].val = cos(a[l].val);
        if (a[i].op == "exp") a[i].val = exp(a[l].val);
        if (a[i].op == "log") {
            if (a[l].val <= 0) throw runtime_error("log domain error");
            a[i].val = log(a[l].val);
        }
        if (a[i].op == "relu") a[i].val = max(0.0, a[l].val);
    } else {
        throw runtime_error("unknown op");
    }
}

a[n].grad = 1.0;
for (int i = n; i >= 1; i--) {
    double g = a[i].grad;
    if (a[i].op == "add") {
        a[a[i].l].grad += g;
        a[a[i].r].grad += g;
    } else if (a[i].op == "sub") {
        a[a[i].l].grad += g;
        a[a[i].r].grad -= g;
    } else if (a[i].op == "mul") {
        int l = a[i].l, r = a[i].r;
        a[l].grad += g * a[r].val;
        a[r].grad += g * a[l].val;
    } else if (a[i].op == "div") {
        int l = a[i].l, r = a[i].r;
        a[l].grad += g / a[r].val;
        a[r].grad -= g * a[l].val / (a[r].val * a[r].val);
    } else if (a[i].op == "sin") {
        int l = a[i].l;

```

```

        a[l].grad += g * cos(a[l].val);
    } else if (a[i].op == "cos") {
        int l = a[i].l;
        a[l].grad -= g * sin(a[l].val);
    } else if (a[i].op == "exp") {
        int l = a[i].l;
        a[l].grad += g * a[i].val;
    } else if (a[i].op == "log") {
        int l = a[i].l;
        a[l].grad += g / a[l].val;
    } else if (a[i].op == "relu") {
        int l = a[i].l;
        if (a[l].val > 0) a[l].grad += g;
    }
}

cout << fixed << setprecision(6);
cout << a[n].val << '\n';
for (int i = 0; i < (int)vars.size(); i++) {
    if (i > 0) cout << ' ';
    cout << a[vars[i]].grad;
}
cout << '\n';

return 0;
}

```

调用示例:

// $f(x,y)=x*y+\sin(x)$, 节点 n 是最终输出, 反向传播后 var 节点上就是偏导。

常见坑:

- 反向模式是从输出往输入传梯度, 输出节点初始梯度为 1。
- 同一个节点可能被多个后续节点使用, 梯度要累加。
- 所有依赖必须已经前向算好, 所以输入节点顺序要是拓扑序。
- ReLU 在 0 处不可导, 本模板按 0 处理。
- 浮点答案一般输出 6 到 10 位小数。

暴力/部分分替代:

- 不会自动求导: 对小数据用数值差分 $f(x+eps)-f(x-eps)$, 但精度较差。
- 只支持四则运算也能拿部分分。
- 若题目给表达式字符串, 先用 SIM-03 建 AST, 再把 AST 转成计算图。

最小测试样例:

输入

```

5
var 2
var 3
mul 1 2
sin 1
add 3 4

```

输出

```

6.909297
2.583853 2.000000

```